

EE115B *Digital Circuits*

Instructor: Chenxi Xiao

Part of slides are from

© Pearson Education

Hengzhao Yang

Zhifeng Zhu

Yajun Ha



上海科技大学
ShanghaiTech University

Combinational Logic

Adders

“Adders are important in computers and also in other types of digital systems in which numerical data are processed.”

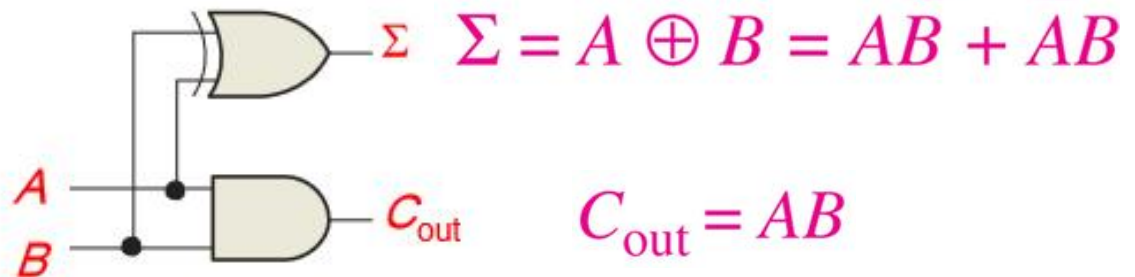
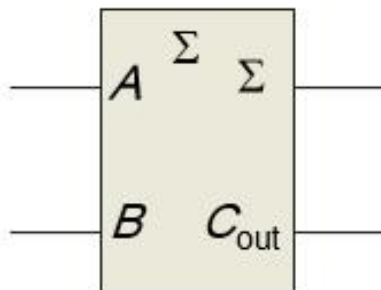
Half-Adder

Basic rules of binary addition are performed by a **half adder**, which has two binary inputs (A and B) and two binary outputs (Carry out and Sum).

The inputs and outputs can be summarized on a truth table.

Inputs		Outputs	
A	B	C_{out}	Σ
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

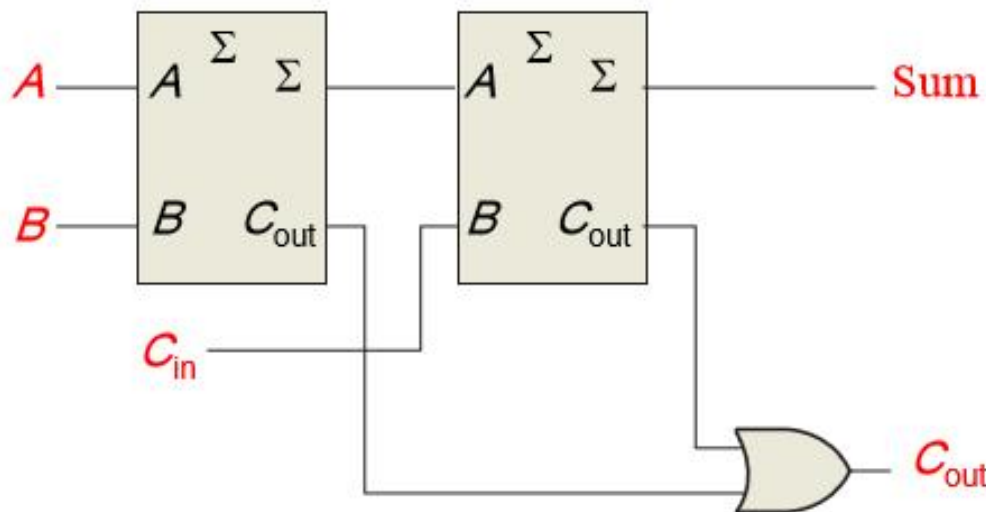
The logic symbol and equivalent circuit are:



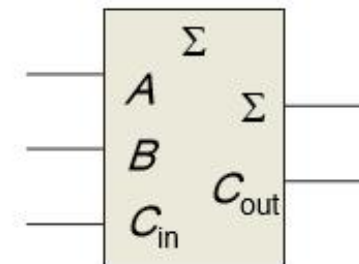
Full-Adder

By contrast, a **full adder** has three binary inputs (A , B , and Carry in) and two binary outputs (Carry out and Sum). The truth table summarizes the operation.

A full-adder can be constructed from two half adders as shown:



Inputs			Outputs	
A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Symbol



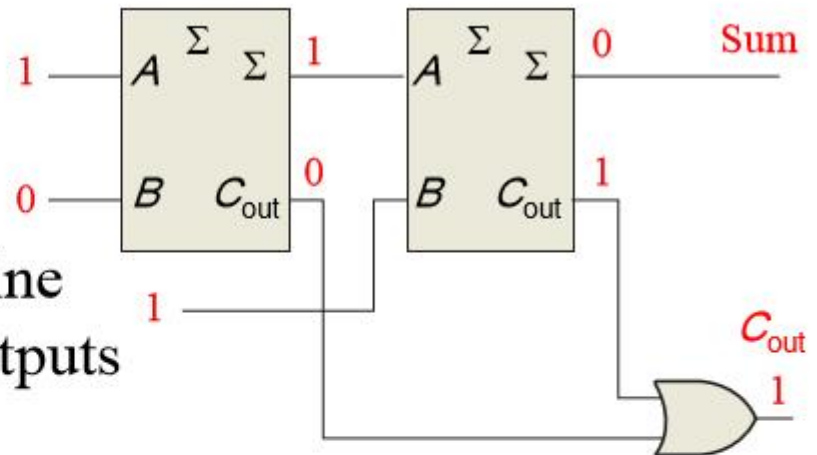
Full-Adder: Exercise

Consider a half adder:

$$\Sigma = A \oplus B = A\bar{B} + \bar{A}B$$

Example $C_{out} = AB$

For the given inputs, determine the intermediate and final outputs of the full adder.



Solution The first half-adder has inputs of 1 and 0; therefore the Sum = 1 and the Carry out = 0.

The second half-adder has inputs of 1 and 1; therefore the Sum = 0 and the Carry out = 1.

The OR gate has inputs of 1 and 0, therefore the final carry out = 1.

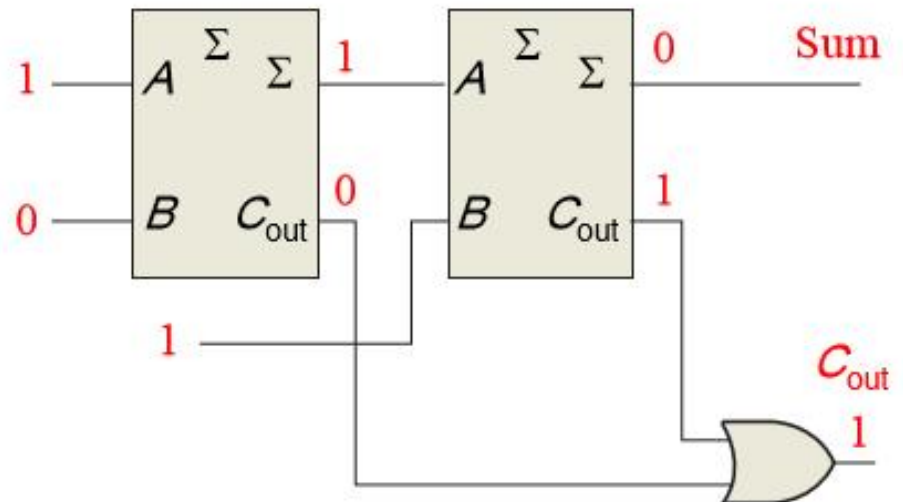


Full-Adder: Exercise

Notice that the result from the previous example can be read directly on the truth table for a full adder.

What does this imply?

Inputs			Outputs	
A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Full-Adder: output functions

$$\Sigma = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

$$= (\bar{A}\bar{B} + AB)C_{in} + (\bar{A}B + A\bar{B})\bar{C}_{in}$$

$$= (\overline{A \oplus B})C_{in} + (A \oplus B)\bar{C}_{in}$$

$$= (A \oplus B) \oplus C_{in}$$

$$C_{out} = \bar{A}B C_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in}$$

$$= (\bar{A}B + A\bar{B})C_{in} + AB(\bar{C}_{in} + C_{in})$$

$$= (A \oplus B)C_{in} + AB$$

TABLE 6-2

Full-adder truth table.

A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

C_{in} = input carry, sometimes designated as CI

C_{out} = output carry, sometimes designated as CO

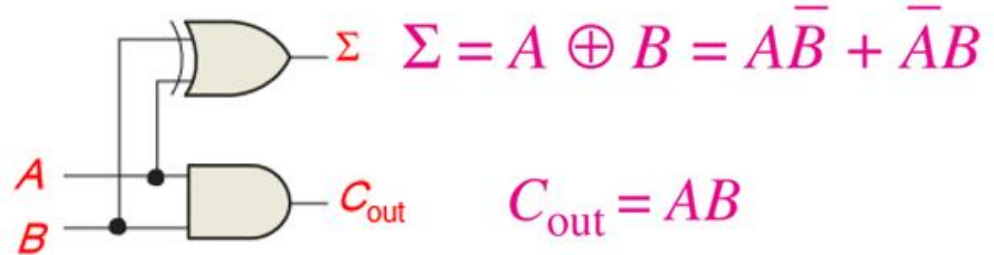
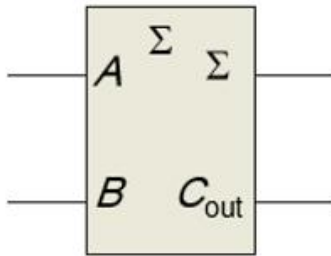
Σ = sum

A and B = input variables (operands)

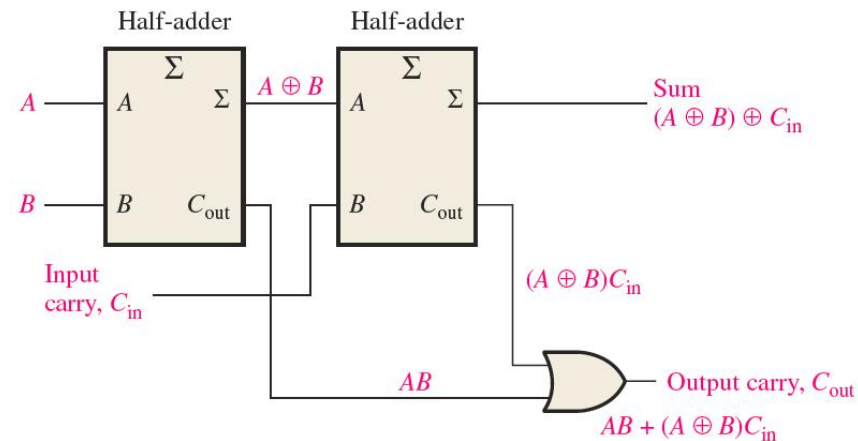
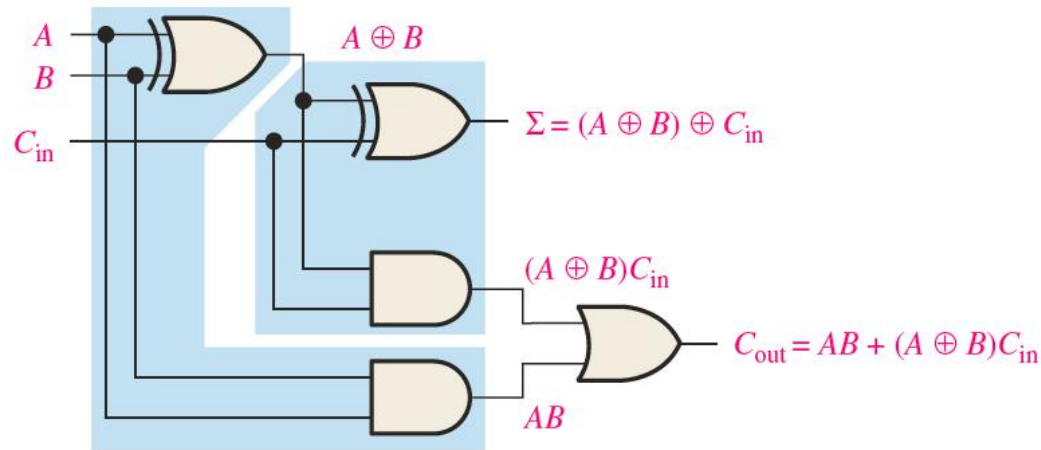


Full-Adder: implementation

Half adder

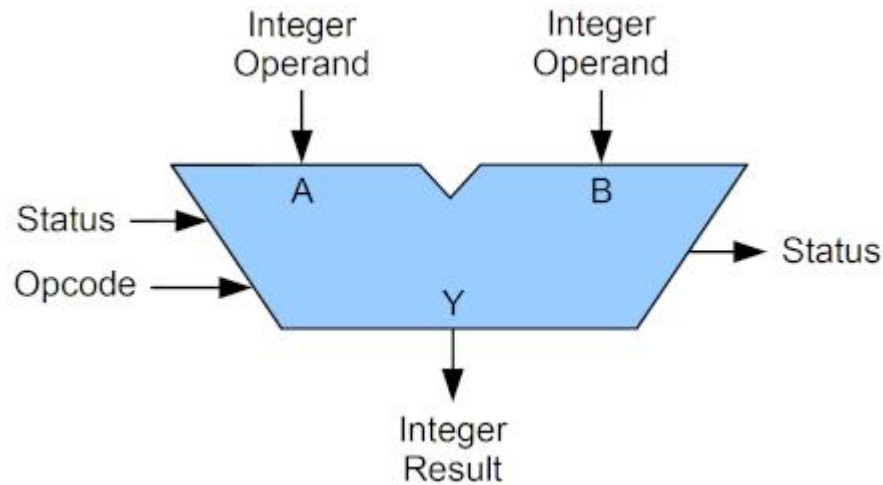


Full adder



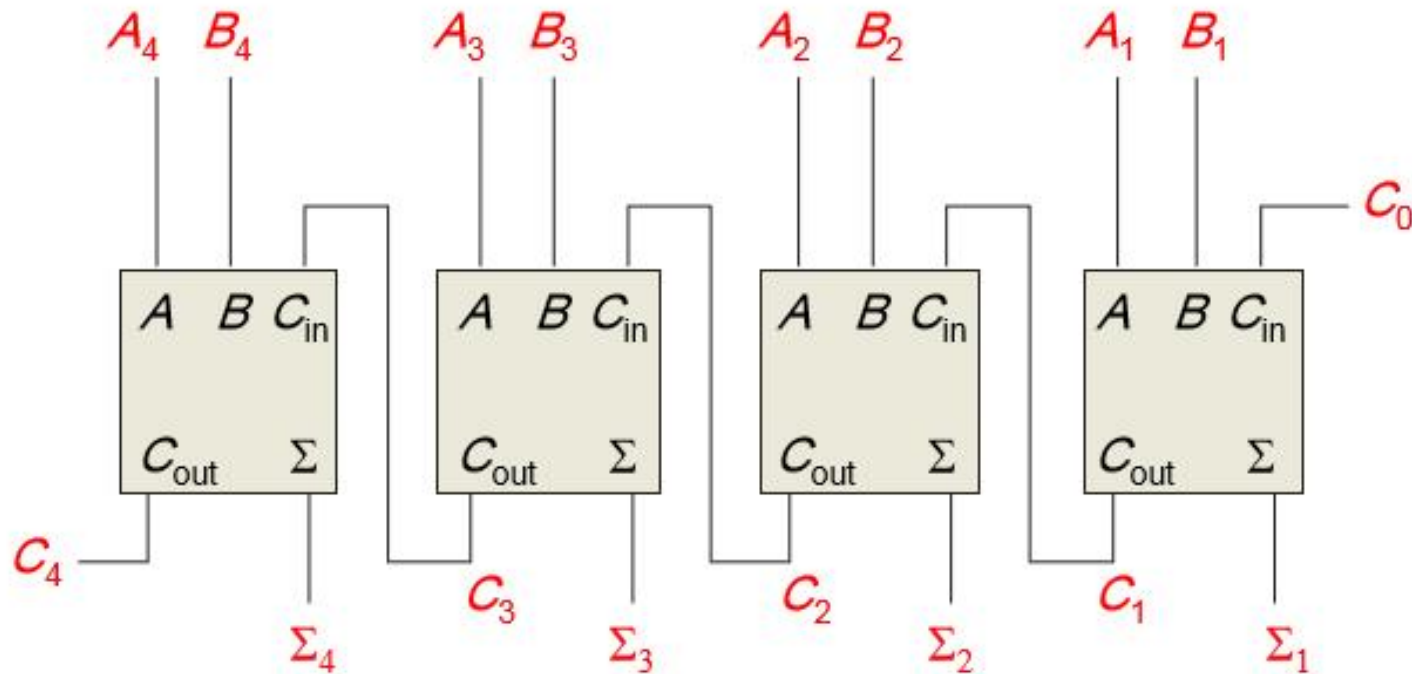
Parallel Adders

How to perform add operation on **integer** binary numbers?



Parallel Adders: Ripple Carry

Full adders are combined into parallel adders that can add binary numbers with multiple bits. A 4-bit adder is shown.

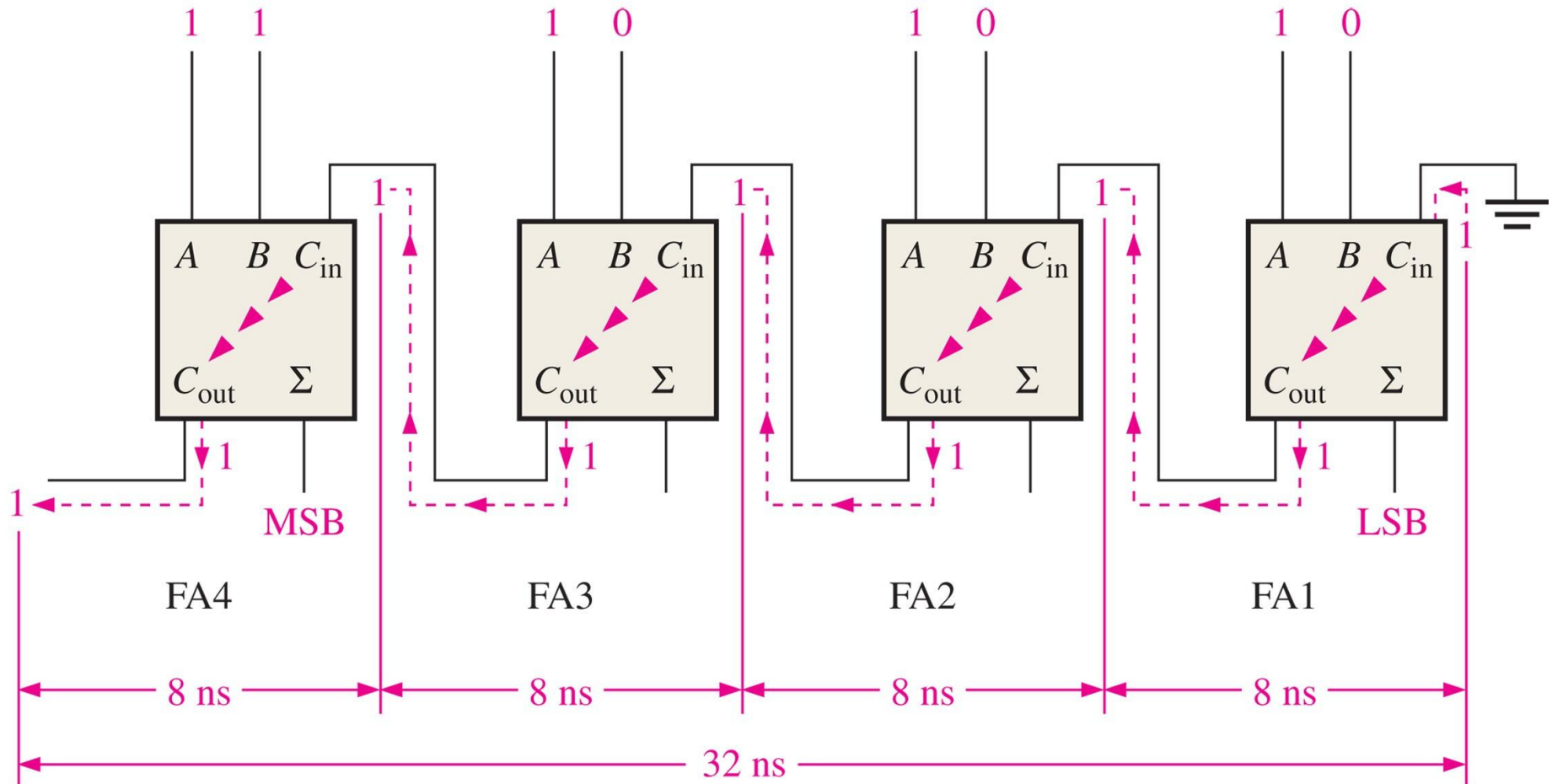


The output carry (C_4) is not ready until it propagates through all of the full adders. This is called *ripple carry*, delaying the addition process.



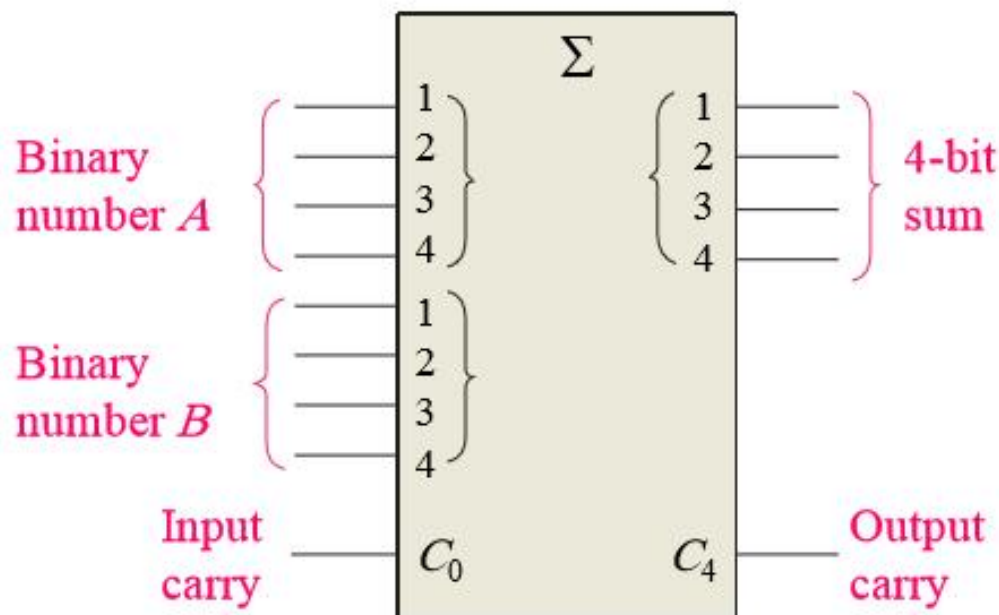
Parallel Adders: Ripple Carry

A 4-bit parallel ripple carry adder showing “worst-case” carry propagation delays.



Parallel Adders: Look-Ahead Carry

The logic symbol for a 4-bit parallel adder is shown. This 4-bit adder includes a carry in (labeled C_0) and a Carry out (labeled C_4).



The 74LS283 is an example. It features *look-ahead carry*, which adds logic to minimize the output carry delay. For the 74LS283, the maximum delay to the output carry is 17 ns.



Take a Closer Look at Output Carry

- **Generated Carry** occurs when an output carry is produced internally

$$C_g = AB$$

- **Propagated Carry** occurs when input carry is rippled to become output carry

$$C_p = A + B$$

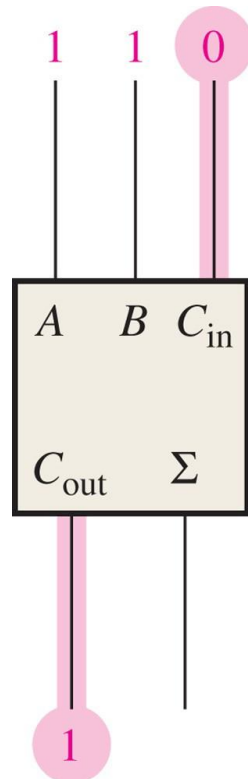
- Output carry

$$C_{out} = C_g + C_p C_{in}$$

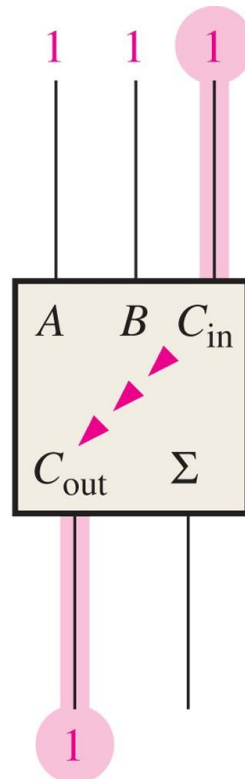


Take a Closer Look at Output Carry

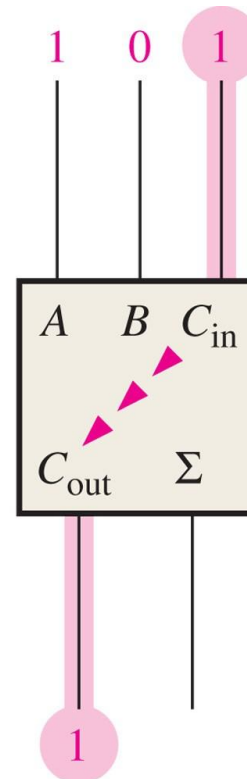
Illustration of conditions for carry generation and carry propagation.



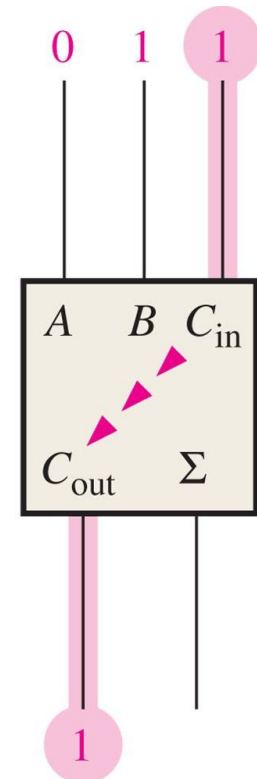
Generated
carry



Propagated carry/
Generated carry



Propagated
carry



Propagated
carry



Exercise

Prove:

$$\begin{aligned}C_{out} &= \bar{A}B C_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in} \\&= (\bar{A}B + A\bar{B})C_{in} + AB(\bar{C}_{in} + C_{in}) \\&= (A \oplus B)C_{in} + AB\end{aligned}$$

Are the same as:

$$C_{out} = C_g + C_p C_{in}$$

TABLE 6-2

Full-adder truth table.

A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

C_{in} = input carry, sometimes designated as CI

C_{out} = output carry, sometimes designated as CO

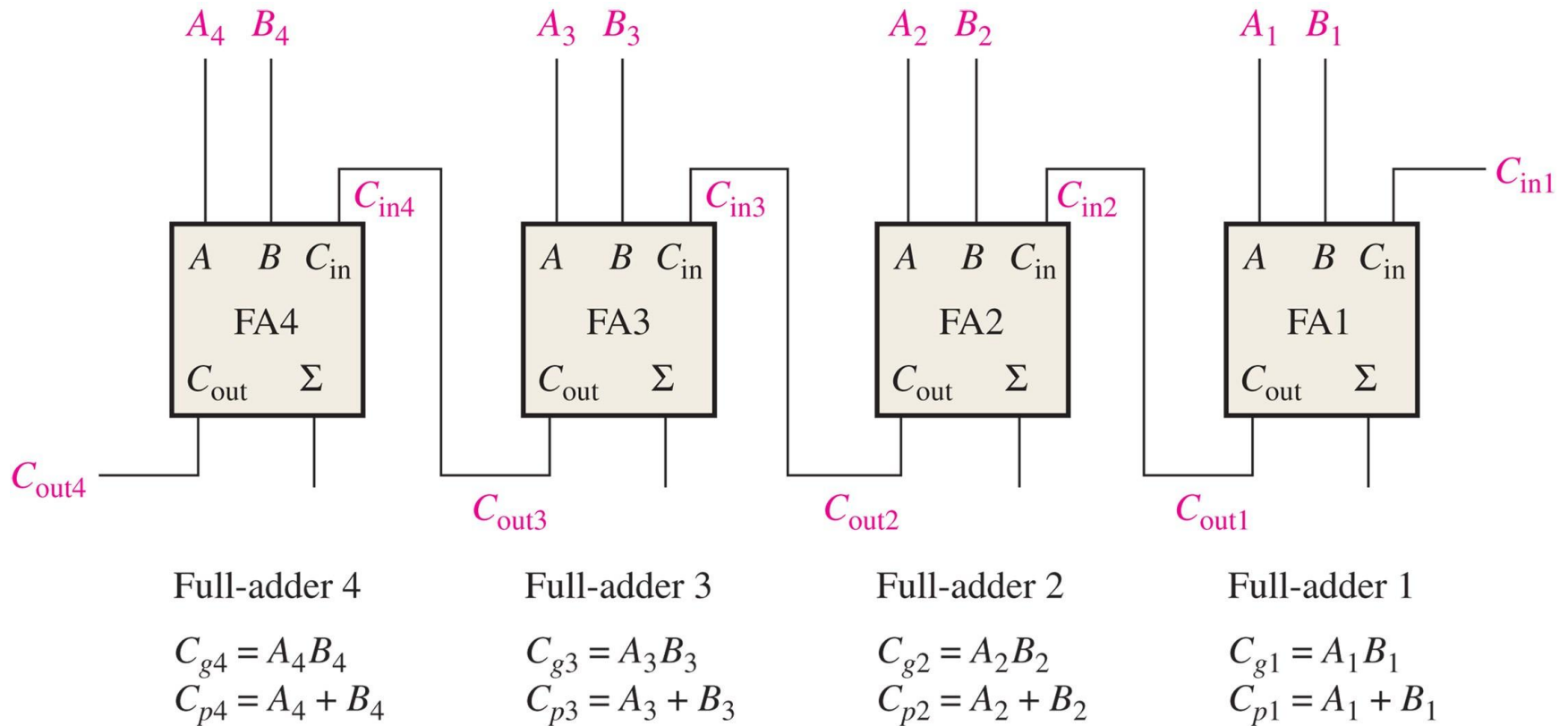
Σ = sum

A and B = input variables (operands)



How to reduce propagation delay?

Carry generation (Cg) and carry propagation (Cp) can be obtained by only very low propagation delay.



How to reduce propagation delay?

Full-adder 1:

$$C_{out1} = C_{g1} + C_{p1}C_{in1}$$

Key insight: represent C_{out} using C_p and C_g , which are of low delay.

Full-adder 2:

$$C_{in2} = C_{out1}$$

$$\begin{aligned} C_{out2} &= C_{g2} + C_{p2}C_{in2} = C_{g2} + C_{p2}C_{out1} = C_{g2} + C_{p2}(C_{g1} + C_{p1}C_{in1}) \\ &= C_{g2} + C_{p2}C_{g1} + C_{p2}C_{p1}C_{in1} \end{aligned}$$

Full-adder 3:

$$C_{in3} = C_{out2}$$

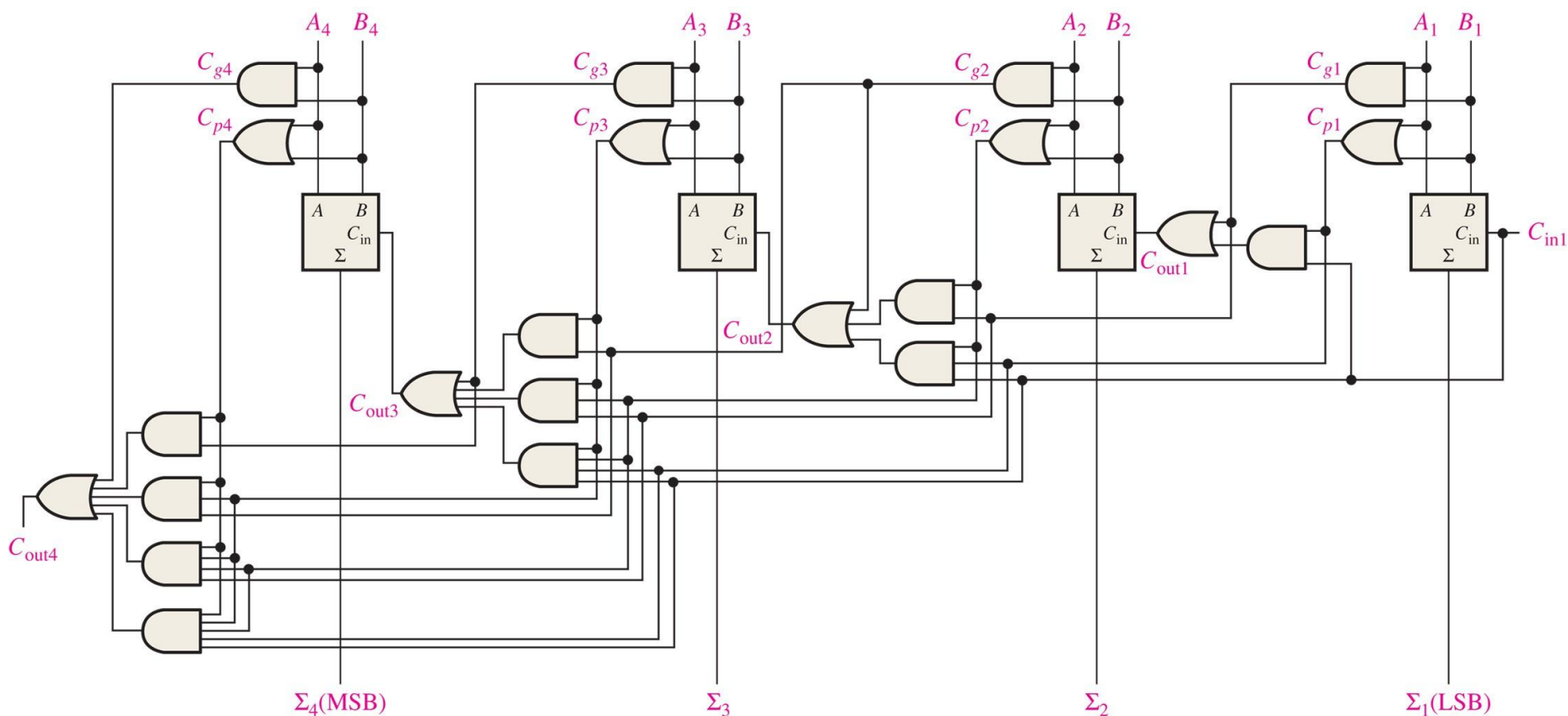
$$\begin{aligned} C_{out3} &= C_{g3} + C_{p3}C_{in3} = C_{g3} + C_{p3}C_{out2} = C_{g3} + C_{p3}(C_{g2} + C_{p2}C_{g1} + C_{p2}C_{p1}C_{in1}) \\ &= C_{g3} + C_{p3}C_{g2} + C_{p3}C_{p2}C_{g1} + C_{p3}C_{p2}C_{p1}C_{in1} \end{aligned}$$

Full-adder 4:

$$C_{in4} = C_{out3}$$

$$\begin{aligned} C_{out4} &= C_{g4} + C_{p4}C_{in4} = C_{g4} + C_{p4}C_{out3} \\ &= C_{g4} + C_{p4}(C_{g3} + C_{p3}C_{g2} + C_{p3}C_{p2}C_{g1} + C_{p3}C_{p2}C_{p1}C_{in1}) \\ &= C_{g4} + C_{p4}C_{g3} + C_{p4}C_{p3}C_{g2} + C_{p4}C_{p3}C_{p2}C_{g1} + C_{p4}C_{p3}C_{p2}C_{p1}C_{in1} \end{aligned}$$

Output carry: Circuit Implementation

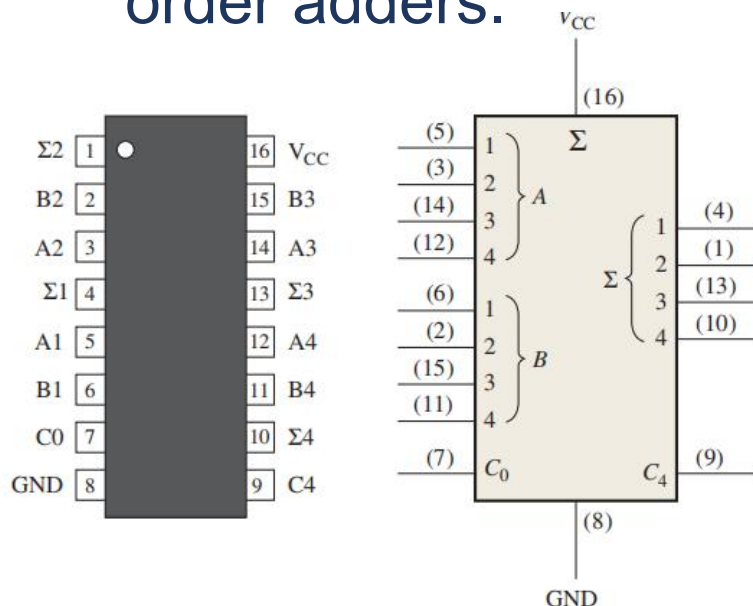


Estimate: How many AND gate does a 64 bit adder needs?



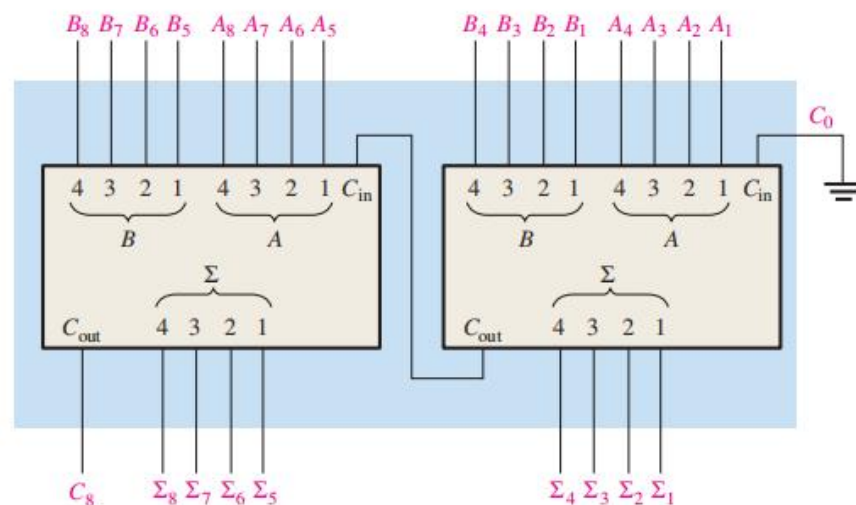
Parallel Adder: 74HC283

4 bit parallel adder. Can be cascaded to expand to high-order adders.



(a) Pin diagram

(b) Logic symbol

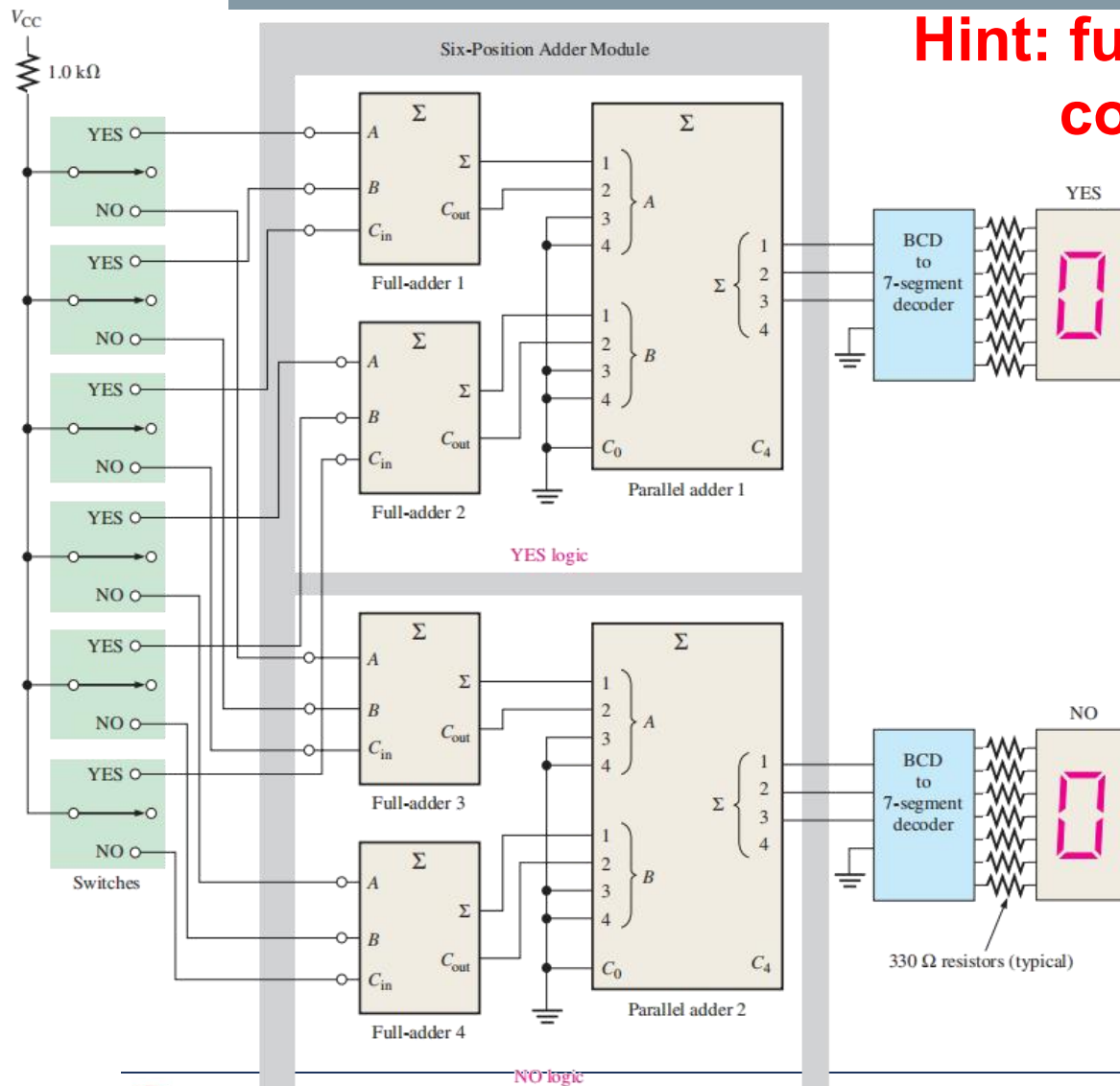


- Lookahead carriers are costly to implement for high order adders.
- Ripple carriers and Look-ahead carriers could be combined.

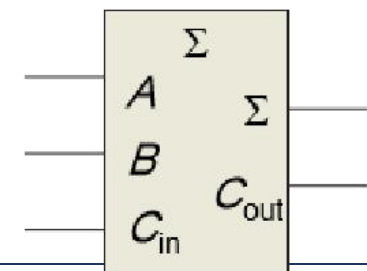


Application: Low-cost Voting System

Hint: full adder can be used to count number of 1s.



Inputs			Outputs	
A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Symbol

Adder is a part of arithmetic logic unit (ALU).
To have ALU, we need additional modules:

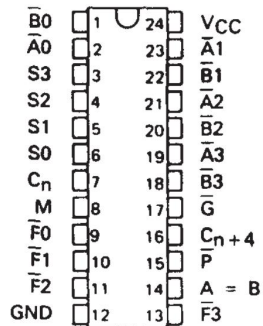
- 1) opcode for +, -, and bitwise operations: xor or and
- 2) optimize for circuit complexity, power consumption, propagation delay, cost and size.



Arithmetic Logic Unit (ALU)

SN54LS181, SN54S181 . . . J OR W PACKAGE
SN74LS181, SN74S181 . . . DW OR N PACKAGE

(TOP VIEW)



<https://www.ti.com/lit/ds/symlink/sn54ls181.pdf>

SELECTION				ACTIVE-LOW DATA		
				M = H LOGIC FUNCTIONS	M = L; ARITHMETIC OPERATIONS	
S3	S2	S1	S0		Cn = L (no carry)	Cn = H (with carry)
L	L	L	L	$F = \overline{A}$	F = A MINUS 1	F = A
L	L	L	H	$F = \overline{AB}$	F = AB MINUS 1	F = AB
L	L	H	L	$F = \overline{A} + B$	F = $\overline{AB}$ MINUS 1	F = $\overline{AB}$
L	L	H	H	F = 1	F = MINUS 1 (2's COMP)	F = ZERO
L	H	L	L	$F = \overline{A + B}$	F = A PLUS (A + $\overline{B}$)	F = A PLUS (A + $\overline{B}$) PLUS 1
L	H	L	H	$F = \overline{B}$	F = AB PLUS (A + $\overline{B}$)	F = AB PLUS (A + $\overline{B}$) PLUS 1
L	H	H	L	$F = A \oplus B$	F = A MINUS B MINUS 1	F = A MINUS B
L	H	H	H	$F = A + \overline{B}$	F = A + $\overline{B}$	F = (A + $\overline{B}$) PLUS 1
H	L	L	L	$F = \overline{AB}$	F = A PLUS (A + B)	F = A PLUS (A + B) PLUS 1
H	L	L	H	$F = A \oplus B$	F = A PLUS B	F = A PLUS B PLUS 1
H	L	H	L	F = B	F = $\overline{AB}$ PLUS (A + B)	F = $\overline{AB}$ PLUS (A + B) PLUS 1
H	L	H	H	F = A + B	F = (A + B)	F = (A + B) PLUS 1
H	H	L	L	F = 0	F = A PLUS A [‡]	F = A PLUS A PLUS 1
H	H	L	H	$F = \overline{AB}$	F = AB PLUS A	F = AB PLUS A PLUS 1
H	H	H	L	F = AB	F = $\overline{AB}$ PLUS A	F = $\overline{AB}$ PLUS A PLUS 1
H	H	H	H	F = A	F = A	F = A PLUS 1

[‡]Each bit is shifted to the next more significant position.



Arithmetic Logic Unit (ALU)

An ALU is usually implemented either as a stand-alone [integrated circuit](#) (IC), such as the 74181, or as part of a more complex IC. In the latter case, an ALU is typically instantiated by synthesizing it from a description written in [VHDL](#), [Verilog](#) or some other [hardware description language](#). For example, the following VHDL code describes a very simple 8-bit ALU:

```
entity alu is
port (  -- the alu connections to external circuitry:
  A  : in signed(7 downto 0);  -- operand A
  B  : in signed(7 downto 0);  -- operand B
  OP : in unsigned(2 downto 0); -- opcode
  Y  : out signed(7 downto 0)); -- operation result
end alu;

architecture behavioral of alu is
begin
  case OP is  -- decode the opcode and perform the operation:
    when "000" => Y <= A + B;  -- add
    when "001" => Y <= A - B;  -- subtract
    when "010" => Y <= A - 1;  -- decrement
    when "011" => Y <= A + 1;  -- increment
    when "100" => Y <= not A;  -- 1's complement
    when "101" => Y <= A and B; -- bitwise AND
    when "110" => Y <= A or B;  -- bitwise OR
    when "111" => Y <= A xor B; -- bitwise XOR
    when others => Y <= (others => 'X');
  end case;
end behavioral;
```

